

Integrating Adobe Experience Manager (AEM) Experience Fragments (XFs) with a headless React frontend involves leveraging AEM's JSON export capabilities and standard APIs to fetch and render content on the client side [1]. The best practice is to **treat AEM as a content repository** and delivery service, while the React application handles presentation and routing . [🔗](#)

Key Best Practices

1. Use AEM's JSON Export:

- AEM components and XFs automatically support a JSON representation through the AEM Content Services Sling Model. This allows you to fetch structured data (content and metadata) for a specific XF variation using a simple API request [1].
- The typical endpoint URL for an XF will look like: `https://[AEM_Instance]/content/experience-fragments/[folder]/[xf-name]/[variation].model.json` [1].

2. Employ the AEM Headless SDKs (Recommended):

- Utilize the dedicated AEM Headless SDKs (JS SDK) to simplify data fetching. These SDKs handle authentication, provide structured data access, and abstract the complexities of the API endpoints [1].
- The SDKs allow you to query content using GraphQL if you are also using Content Fragments, or simply fetch the raw XF JSON.

3. Client-Side Rendering in React:

- Fetch the XF JSON data within your React components (e.g., using `useEffect` or a data-fetching library like React Query).
- Parse the returned JSON and dynamically render the content using your own corresponding React components. You will need a mapping between AEM component types (e.g., `text` , `image`) and your React components [1].
- Libraries like the AEM SPA Editor JS SDK can help with this mapping, although a manual mapping is also feasible for simpler integrations [2].

4. Handle Dynamic Content and Links:

- AEM delivers raw data. Ensure your React components correctly handle and style the HTML output for text components, and use the provided URLs for images and internal links [2].
- Pay attention to relative paths for assets; they may need to be prefixed with your AEM instance URL, depending on your configuration.

5. Enable In-Context Preview:

- To allow authors to use the AEM visual editor, you need to set up a preview environment. This typically involves configuring your React app to run within an iframe in the AEM editor [3].
- The AEM SPA Editor framework helps facilitate this, enabling authors to click and edit components directly, which communicates back to the React app to update content dynamically [2, 3].

6. Optimize Performance with CDN and Caching:

- Implement a Content Delivery Network (CDN) in front of your AEM publish environment to cache XF JSON responses and assets [1]. This is crucial for performance as the frontend makes API calls to fetch content.
- Use client-side caching strategies in your React app to minimize repeated network requests. 

By following these practices, you can establish a robust, maintainable, and high-performing integration between the content authoring power of AEM Experience Fragments and the flexibility of a modern headless React frontend [1, 2].

Useful Resources

- AEM Headless Tutorials for detailed guides and examples [3].
- AEM SPA Editor JS SDK Documentation for leveraging in-context editing capabilities [2].